SQL Queries

1.

```
SELECT DISTINCT IncidentType FROM mpdcrimedata;
```

2.

```
SELECT DATE(CFSDate) AS crimeDay, COUNT(IncidentType) AS property_counts

FROM mpdcrimedata

WHERE

IncidentType LIKE '%LARCENY THEFT – MOTOR VEHICLE%' OR IncidentType LIKE '%LARCENY THEFT%' OR IncidentType LIKE '%BURGLARY%' OR IncidentType LIKE '%ARSON%'

AND

`ZIP Code` LIKE '330%' OR `ZIP Code` LIKE '331%' OR `ZIP Code` LIKE '332%'

`ZIP Code` IN('33133', '33146', '33143', '33173', '33183', '33193', '33156', 33196', '33176', '33186', '33158', '33157', '33177', '33187', '33189', '33190', '33170', '33032', '33031', '33039', '33033', '33030', '33035', '33034')

AND YEAR(CFSDate) = 2024

GROUP BY DATE(CFSDate)

ORDER BY crimeDay;
```

3.

```
SELECT STR_TO_DATE(CONCAT(BEGIN_YEARMONTH, LPAD(BEGIN_DAY, 2, '0')), '%Y%m%d') AS stormDay, COUNT(EVENT_TYPE) AS storm_counts

FROM stormevents2024

WHERE CZ_FIPS = 086

GROUP BY STR_TO_DATE(CONCAT(BEGIN_YEARMONTH, LPAD(BEGIN_DAY, 2, '0')), '%Y%m%d')

ORDER BY stormDay;
```

PYTHON:

STEP 1 IMPORT:

```
#import packages
import mysql.connector

import sqlalchemy
import pandas as pd
from datetime import datetime
import matplotlib.pyplot as plt
```

STEP 2 Create Connection:

```
#create connection to mySQL database
cnx = mysql.connector.connect(user="root", host='127.0.0.1', port='3306',
        password="Student1", database="dat375")
#provide the queries to select the data from the mySQL database and correct table
cursor = cnx.cursor()
#read in data from mySQL database using the query created and the connection created
cursor.execute("SHOW TABLES;")
#inspect data to confirm data was read in properly
tables=cursor.fetchall()

for table in tables:
        print(table[0])

#confirm python/sql queries
query1=""
```

```
cursor.execute(query1)
results = cursor.fetchall()

query2=""
cursor.exeute(query2)
results = cursor.fetchall()

#convert query to data frame
crimes_df = pd.read_sql(query1, cnx)
storms_df= pd.read_sql(query2, cnx)
cnx.close()
```

STEP 3 CLEAN DATA:

```
#creating new data frame with all of the dates included
def storms_df(year):
        start_date = f"2024-01-01"
        end_date=f"2024-12-31"

        all_dates = pd.date_range(start=start_date, end=end_date, freq='D')
        storms_df = pd.DataFrame({'stormDay': all_dates})

        return storms_df
dates_df = storms_df(2024)
print(dates_df)

#printing original data frame with storm counts
```

```
print(storms_df)


#creating final merged data frame with dates for 2024 and filled in missing zero values for non-storm days

storms_df['stormDay']=pd.to_datetime(storms_df['stormDay'])

dates_df['stormDay']=pd.to_datetime(dates_df['stormDay'])

smerge_df = pd.merge(storms_df, dates_df, on='stormDay', how='right')

smerge_df = smerge_df.fillna(0)

print(smerge_df)


#No Storm Days table

no_storms = smerge_df[smerge_df['storm_counts'] == 0]

print(no_storms)


#Storm Days table

yes_storms = smerge_df[smerge_df['storm_counts'] > 0]

print(yes_storms)
```

STEP 4 DATA ANALYSIS: (p-value)

STEP 5 VISUAL:

STEP 6 SAVE:

```
#save data to a .csv

merged_df = pdDataFrame(data)

merged_df.to_csv('DAT 375 Project Two_Visualization_Alyssa.csv', index=False)

print("CSV done")
```

```
#save the visualization to a .png

Plt.savefig('DAT 375 Project Two_Visualization_Alyssa.png')
```